

21. Regulární jazyky a jejich modely (konečné automaty, regulární výrazy)

21. Regulární jazyky a jejich modely

SZZ okruh 21 (FIT BUT). Nejjednodušší třída Chomského hierarchie — tři ekvivalentní modely.

Shrnutí

Jazyky a regulární jazyk

- **Abeceda** Σ (konečná množina symbolů), **řetězec** (i prázdný ϵ), **jazyk** $L \subseteq \Sigma^*$.
- Operace nad jazyky: sjednocení, průnik, rozdíl, doplněk, **konkatenace**, **iterace** (*), reverzace.
- **Regulární jazyk** je ekvivalentně: přijímán **konečným automatem**, značen **regulárním výrazem**, generován **regulární gramatikou**.

Konečné automaty

- KA = pětice **M** = (**Q**, Σ , **R**, **s**, **F**).
- **NFA** (ϵ -přechody, více přechodů na stejný symbol) $\times$ **DFA** (max. 1 přechod) $\times$ **úplný DFA** $\times$ **minimální DFA**.
- **Determinizace** (NFA $\rightarrow$ DFA) podmnožinovou (powerset) konstrukcí: až 2^n stavů.

Regulární výrazy a gramatiky

- RV s operátory ****konkatenace** ($\cdot$), **sjednocení** (+), **iterace** (*); priorita $* > \cdot > +$.
- **Regulární gramatika** (N,T,P,S) — pravá/levá, generuje regulární jazyk; převoditelná na/z KA.
- **Pumping lemma** — dokazuje (sporem), že jazyk **NENÍ** regulární (nikdy ne že je).

Navazuje na [[okruhy/22-bezkontextove-jazyky]] (silnější třída), automaty viz [[okruhy/03-sekvencni-logicke-obvody]], využití v [[okruhy/23-struktura-prekladace|lexikální analýze]].

Související syntéza

- [[synthesis/konecne-automaty-napric-obory|Konečné automaty napříč obory]] — syntéza

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Konečné automaty $\square$ [[#Konečné automaty]] - *Formální definice KA?* $\square$ Pětice (Q, Σ, R, s, F) — stavy, abeceda, pravidla, počáteční a koncové stavy. - *DFA vs. NFA?* $\square$ NFA: ϵ -přechody a více přechodů na stejný symbol; DFA: nejvýše jeden přechod. - *Determinizace NFA $\rightarrow$ DFA?* $\square$ Podmnožinová (powerset)

konstrukce, max. 2^n stavů; každý NFA má ekvivalentní DFA. - *Minimalizace?* $\square$ Sloučení nerozlišitelných stavů $\square$ jednoznačný minimální DFA.

Regulární výrazy $\square$ [[#Regulární výrazy a gramatiky]] - *Operace RV?* $\square$ konkatenace, sjednocení (+), iterace (); *příklad* $a(a+b)b$. - *Převod RV $\square$ KA?* $\square$ Vzájemně převoditelné (RV $\square$ NFA $\square$ DFA).

Regulární jazyky $\square$ [[#Jazyky a regulární jazyk]] - *Jazyk, slovo, abeceda?* $\square$ Abeceda Σ ; slovo $\square \Sigma$; jazyk $L \subseteq \Sigma^*$. - *Důkaz neregulárnosti?* $\square$ Pumping lemma (sporem).

Gramatiky $\square$ [[#Regulární výrazy a gramatiky]] - *Regulární gramatika vs. RV?* $\square$ RG generuje, RV značí týž regulární jazyk; Chomského hierarchie (RJ $\square$ BKJ $\square \dots$).

Plné znění (ke studiu)

Abeceda

Konečná, neprázdná množina prvků, které nazýváme symboly. Značíme Σ .

Řetězec

Jakákoliv posloupnost terminálních a neterminálních symbolů (znaků).

Nechť Σ je abeceda.

- ϵ je řetězec nad abecedou Σ .
 - ϵ : Prázdný řetězec (prázdné slovo), který ale vyhovuje jazyku (je možné jej vygenerovat a přijmout).
- Pokud x je řetězec nad abecedou Σ a $s \in \Sigma$, pak xs je **řetězec** nad abecedou Σ .

Jazyk

Nechť Σ^* značí **množinu** všech **řetězců (slov)** nad Σ . Každá **podmnožina** $L \subseteq \Sigma^*$ je **jazyk** nad Σ . Jazyk tedy značí podmnožinu řetězců abecedy. **Počet** všech **slov** jazyka je jeho **kardinalita**.

- **Konečný a nekonečný** - Jazyk L je **konečný**, pokud L obsahuje **konečný počet řetězců**, jinak je **nekonečný**.
- **Operace nad jazyky**
 - **Sjednocení** - Stejně jako u množin.
 - **Průnik** - Stejně jako u množin.
 - **Rozdíl** - Stejně jako u množin.
 - **Doplňěk** - Stejně jako u množin.
 - **Konkatenace** - Nechť L_1 a L_2 jsou dva **jazyky** nad Σ . Konkatenace jazyků L_1 a L_2 je definována jako

$L = L_1 \bullet L_2 = \{xy : x \in L_1 \text{ a } y \in L_2\}$. Například

$L = L_1 \bullet L_2 = \{0, 1\} \bullet \{2, 3\} = \{02, 03, 12, 13\}$.

- **Pozor:** Stejně jako při násobení můžeme konkatenací operátor ($\bullet$) někdy vynechat: $L_1 \bullet L_2 = L_1 L_2$.
- **Pozor:** $L\{\epsilon\} = \{\epsilon\}L = L$, ale $\emptyset L = L\emptyset = \emptyset$.
- **Reverzace** - Nechť L je jazyk nad abecedou Σ . Reverzace jazyka L , **reverse(L)**, je definována: **reverse(L) = {reverse(x) : x $\in$ L}**. Jedná se o převrácení slov abecedy. **reverse({02, 03, 12, 13}) = {20, 30, 21, 31}**.

Regulární jazyk

Regulární jazyky jsou **nejjednodušší formální jazyky** v rámci **Chomského hierarchie**. Nad abecedou Σ je lze zavést následovně:

- Prázdný jazyk $\emptyset$ je regulární.
- Pro každé $a, a \in \Sigma$, jazyk $\{a\}$ je regulární.
- Pokud A a B jsou regulární jazyky, $A \cup B$ (sjednocení), $A \cdot B$ (konkatenace), a A^* (iterace, umožňuje jazyk prázdného řetězce $\{\epsilon\}$) také regulární.
- Žádné další regulární jazyky nejsou.

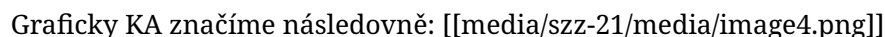
Dále platí, že jazyk je regulární, pokud:

- existuje **konečný automat** (deterministický i nedeterministický), který **akceptuje** právě všechna slova z tohoto jazyka (**akceptuje/přijímá** tento jazyk),
- existuje **regulární výraz**, který tento jazyk **značí**,
- existuje **regulární gramatika**, která jej **generuje**.

Konečný automat (KA)

Konečný automat (KA) je **pětice** $M = (Q, \Sigma, R, s, F)$, kde:

- Q je **konečná množina stavů**,
- Σ je **vstupní abeceda**,
- R je **konečná množina pravidel** tvaru: $pa \rightarrow q$, kde $p, q \in Q$, $a \in \Sigma \cup \{\epsilon\}$,
- $s \in Q$ je **počáteční stav**,
- $F \subseteq Q$ je **množina koncových stavů**.

Graficky KA značíme následovně: 

Přijímaný jazyk KA

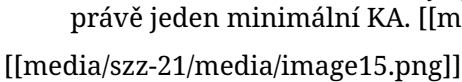
Pokud se s nějakou **vstupní posloupností znaků** dostaneme z **počátečního stavu** automatu až do **koncového stavu** automatu, automat tento **jazyk přijímá**.

Ekvivalentní KA

Dva modely pro popis formálních jazyků (např. konečné automaty) jsou ekvivalentní, pokud specifikují **tentýž jazyk**. (Jsou ekvivalentní, pokud oba dokáží zpracovat všechny řetězce z určitého jazyka a odmítnou všechny řetězce, které z daného jazyka nevycházejí).

Typy konečných automatů

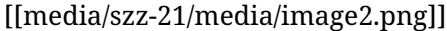
- **Nedeterministický**: Může obsahovat ϵ přechody a existují stavy, ze kterých lze s načtením **stejného symbolu** přejít do **více stavů**.
- **Bez epsilon přechodů** - Neobsahuje ϵ přechody.
- **Deterministický** - Neobsahuje ϵ přechody a z každého **stavu** může přejít **maximálně do jednoho** dalšího s načtením **stejného znaku**.
- **Úplný deterministický** - Pro **každý znak** abecedy existuje **právě jeden** přechod v **každém stavu** (nepotřebné směřují do **false stavu**). **Nemůže se zaseknout**.

- **Dobře specifikovaný** - Nemá nedostupné stavy a má maximálně **jeden neukončující** stav (false stav). Pro každý konečný automat existuje ekvivalentní dobře specifikovaný konečný automat.
- **Minimální** - Pokud obsahuje pouze rozlišitelné stavy. Pro dobře specifikovaný KA existuje právě jeden minimální KA. 

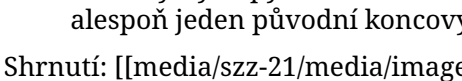


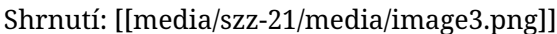
Shrnutí:

Determinizace KA



Převod nedeterministického KA na deterministický. Pokud má nedeterministický KA **n stavů**, má jeho deterministická varianta nejvíce **2n stavů**. Každý nedeterministický automat má svoji deterministickou variantu. Postup:

1. Odstranění ϵ přechodů pomocí ϵ uzávěru - množina stavů, do kterých se můžeme dostat přečtením libovolného symbolu z abecedy.
2. Z počátečního stavu vytváříme nové stavy tak, že tyto stavy jsou množinou stavů, do kterých se můžeme dostat přečtením libovolného symbolu (prázdné stavy neuvažujeme). Z takto vzniklých nových stavů postupujeme dále obdobně (je nutné dávat pozor, aby byly brány v potaz všechny výstupy vzniklé množiny stavů). Za nové koncové stavy označíme ty, které obsahují alespoň jeden původní koncový stav. 

Shrnutí: 

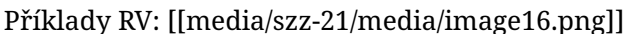
Regulární výraz (RV)



Jedná se o výrazy s operátory “.”, “+”, “*”, které značí v tomto pořadí **konkatenaci**, **sjednocení** a **iteraci**. Nechť Σ je **abeceda**. Regulární výrazy nad abecedou Σ a **jazyky**, které **značí**, jsou definovány následovně:

- $\emptyset$ je RV značící prázdnou množinu (**prázdný jazyk**),
- ϵ je RV značící jazyk $\{\epsilon\}$,
- a , kde $a \in \Sigma$, je RV značící jazyk $\{a\}$,
- Nechť r a s jsou regulární výrazy značící po řadě jazyky L_r a L_s , potom:
 - $(r.s)$ je RV značící jazyk $L = L_r L_s$,
 - $(r+s)$ je RV značící jazyk $L = L_r \cup L_s$,
 - (r^*) je RV značící jazyk $L = L_r^*$.

Závorky lze redukovat zavedením priority operátorů:

Příklady RV: 



Převod regulárního výrazu na konečný automat

[[media/szz-21/media/image14.png]]

Regulární gramatika

Každá **regulární gramatika generuje regulární jazyk**. Je to čtveřice: **(N, T, P, S)**

- N - Konečná množina neterminálních symbolů,
- T - Konečná množina terminálních symbolů,
- P - Konečná množina pravidel, ve tvaru:
 - [[media/szz-21/media/image10.png]] (w je řetězec)
 - [[media/szz-21/media/image5.png]]
 - [[media/szz-21/media/image1.png]] , poté se však nesmí S vyskytovat na pravé straně pravidla.
- S - Počáteční symbol (S náleží do N).

Existují **pravé** ([[media/szz-21/media/image11.png]]) a **levé** ([[media/szz-21/media/image9.png]]) **regulární gramatiky**, které jsou si **ekvivalentní**.

Příklady:

Oproti zápisu výše (N,T,P,S) jsou v zápisu příkladu prohozeny množiny terminálních a neterminálních symbolů ve formě (T,N,P,S). [[media/szz-21/media/image8.png]]

Pumping lemma

[[media/szz-21/media/image6.png]]

Používá se k dokázání, že jazyk **NENÍ REGULÁRNÍ** (důkaz se provádí sporem). **Nemůže** tedy dokázat, že jazyk je regulární. Nechť jazyk **L** je **RJ**. Pak existuje **k** ≥ 1 takové, že: pokud **z** $\in$ **L** a **|z|** $\geq$ **k**, pak existuje **u, v, w**: **z = uvw**,

- **v** $\neq \epsilon$
- **|uv|** $\leq$ **k**
- pro každé **m** ≥ 0 , **uv^mw** $\in$ **L**

Pumping Lemma (For Regular Languages) | Example 1

Odkazy

- Definice z IFJ: IFJ Teorie
- Minimalizace a kanonizace, nedeterministické konečné automaty a determinizace

Zdroje

- SZZ okruh 21 — studijní materiály FIT BUT (szz-21.docx). Obrázky: media/szz-21/.

Navigace: [[index | • Přehled okruhů]] • [[okruhy/20-boolovy-algebry | 20. Boolovy algebry]] • další: [[okruhy/22-bezkontextove-jazyky | 22. Bezkontextové jazyky]]